REAL-TIME US STOCKS DATA PIPELINE

Double-click or double-tap to edit

Team Role Outlines — 6 Members

M1 — Data & Kafka

Kaggle Dataset → Python Producer → Kafka → Consumer Validation

TASKS

- Download & profile Kaggle dataset
- Define canonical JSON event schema
- Assess and clean data quality
- Design micro-batch replay strategy
- Implement Python simulation engine
- Implement Kafka Producer
- Set up Kafka cluster (Docker)
- Create Kafka topics
- Implement Consumer validation script
- Monitor consumer lag & ordering

HANDOVER TO NEXT MEMBER

- Kafka broker address & port
- Topic name(s)
- Consumer group ID for Spark
- Full JSON event schema + sample payload
- Kafka Docker Compose snippet
- Producer config file
- Data quality report

DEFINITION OF DONE

- Schema documented & versioned
- Producer validated — events in Kafka
- Consumer validation confirmed
- Handover package delivered to M2

M2 — Spark & Data Lake

Kafka Topic → Spark Structured Streaming → HDFS / Parquet

TASKS

- Set up Spark cluster (Docker)
- Configure Kafka source connector
- Enforce canonical schema on stream
- Parse & transform streaming events
- Handle invalid events (dead-letter)
- Implement windowed aggregations
- Configure HDFS checkpointing
- Design HDFS directory layout
- Write Parquet with partitioning
- Validate Parquet files & row counts
- Test fault recovery via checkpoint

HANDOVER TO NEXT MEMBER

- HDFS base path for clean Parquet
- HDFS base path for aggregated Parquet
- Parquet schema & partition key structure
- Sample Spark read snippet
- Data completeness report (row counts, date range)
- HDFS namenode address
- Spark cluster Docker service name

DEFINITION OF DONE

- Spark reads Kafka with no message loss
- Schema enforcement active
- Parquet files written & queryable
- Checkpoint recovery verified
- Handover package delivered to M3

M3 — Spark SQL & PostgreSQL

HDFS Parquet → Spark SQL Transforms → PostgreSQL Analytical DB

TASKS

- Design analytical data model (fact/dim)
- Write Spark SQL aggregation transforms
- Compute OHLCV summaries, MAs, gainers/losers

- Design PostgreSQL schema (tables, indexes)
- Implement JDBC incremental load
- Run data quality checks pre/post load
- Create PostgreSQL views for BI & ML

HANDOVER TO NEXT MEMBER

- PostgreSQL host, port, database name
- Read-only credentials (username & password)
- Full table schema & column descriptions
- Data model diagram
- Row counts & date coverage per table
- Recommended views for Power BI
- HDFS Parquet path (for M5 ML)
- Spark SQL job paths (for M4 Airflow)

DEFINITION OF DONE

- Spark SQL transforms validated
- PostgreSQL tables populated
- Incremental load tested
- Handover packages delivered to M4, M5, M6

M4 — Docker, Airflow & GitHub

Full-Stack Containerisation + Orchestration + Documentation

TASKS

- Collect Docker snippets from M1, M2, M3
- Build single docker-compose.yml for all services
- Configure shared Docker network & DNS names
- Manage .env file for all credentials
- Test full stack: docker compose up
- Set up Airflow in Docker
- Build Airflow DAG (Spark SQL jobs → PG load → validation)
- Configure DAG schedule, retries, SLAs
- Manage GitHub repo structure & branching
- Write top-level README & architecture docs
- Collect & organise all handover packages
- Write final demo script

HANDOVER TO NEXT MEMBER

- docker-compose.yml (shared on GitHub main)
- .env.example — all variable names & descriptions
- Docker service hostnames for all components

- Airflow UI URL & admin credentials
- GitHub repo URL & branching conventions

DEFINITION OF DONE

- docker compose up starts all services cleanly
- Airflow DAG runs full cycle successfully
- GitHub repo fully organised
- README & architecture docs complete
- Demo script written & rehearsed

M5 — Power BI & ML

PostgreSQL → Power BI Dashboard + Spark MLlib Model + Streamlit ML Tab

TASKS

- Connect Power BI to PostgreSQL
- Build star schema data model
- Write DAX measures (returns, MAs, volatility, volume)
- Build 3 dashboard pages (market overview, ticker deep-dive, volume)
- Validate DAX figures against SQL
- Write business insights report (PDF)
- Define ML problem & prediction target
- Engineer features from Parquet data
- Train & evaluate Spark MLlib model
- Generate predictions CSV
- Build Streamlit ML predictions tab

HANDOVER TO NEXT MEMBER

- Streamlit app.py base file (for M6 to add RAG tab)
- Predictions CSV path
- ML model evaluation metrics

DEFINITION OF DONE

- Power BI connected & all DAX validated
- Dashboard complete (3+ pages)
- Business insights report written
- ML model trained & evaluated
- Streamlit ML tab working

M6 — RAG AI Assistant

Knowledge Sources → Embeddings → Vector Store → LLM → Streamlit RAG Tab

TASKS

- Curate financial knowledge documents
- Clean & format documents to text/markdown
- Design chunking strategy (size, overlap)
- Generate embeddings (sentence-transformers or OpenAI)
- Index chunks into vector store (FAISS / ChromaDB)
- Test retrieval quality on sample queries
- Build prompt template (system + context + question)
- Integrate LLM (OpenAI / local Ollama)
- Implement full RAG chain (LangChain / LlamaIndex)
- Add source citation to answers
- Evaluate RAG on 20-question test set
- Build Streamlit RAG chat tab
- Merge RAG tab into M5 Streamlit app

HANDOVER TO NEXT MEMBER

- — (final consumer; no downstream handover)

DEFINITION OF DONE

- Vector store indexed & retrieval working
- LLM generating grounded answers with citations
- RAG evaluated on test question set
- Streamlit RAG tab merged with M5 app
- RAG architecture doc written